

Inteligencia Artificial



Inteligencia Artificial

Estrategias de búsqueda no informada

Referencias:

<https://www.youtube.com/watch?v=UAIDAxof3kA>

<https://www.youtube.com/watch?v=bUfkF65GnP4>

<https://www.youtube.com/watch?v=WViscSzNnWs>

REINVENTEMOS
EL FUTURO



Estrategias de búsqueda no informada

En Inteligencia Artificial (IA), las estrategias de búsqueda no informadas, también conocidas como búsquedas a ciegas, son algoritmos de búsqueda que no utilizan ningún conocimiento previo o información heurística sobre el problema para guiar la búsqueda hacia la solución.

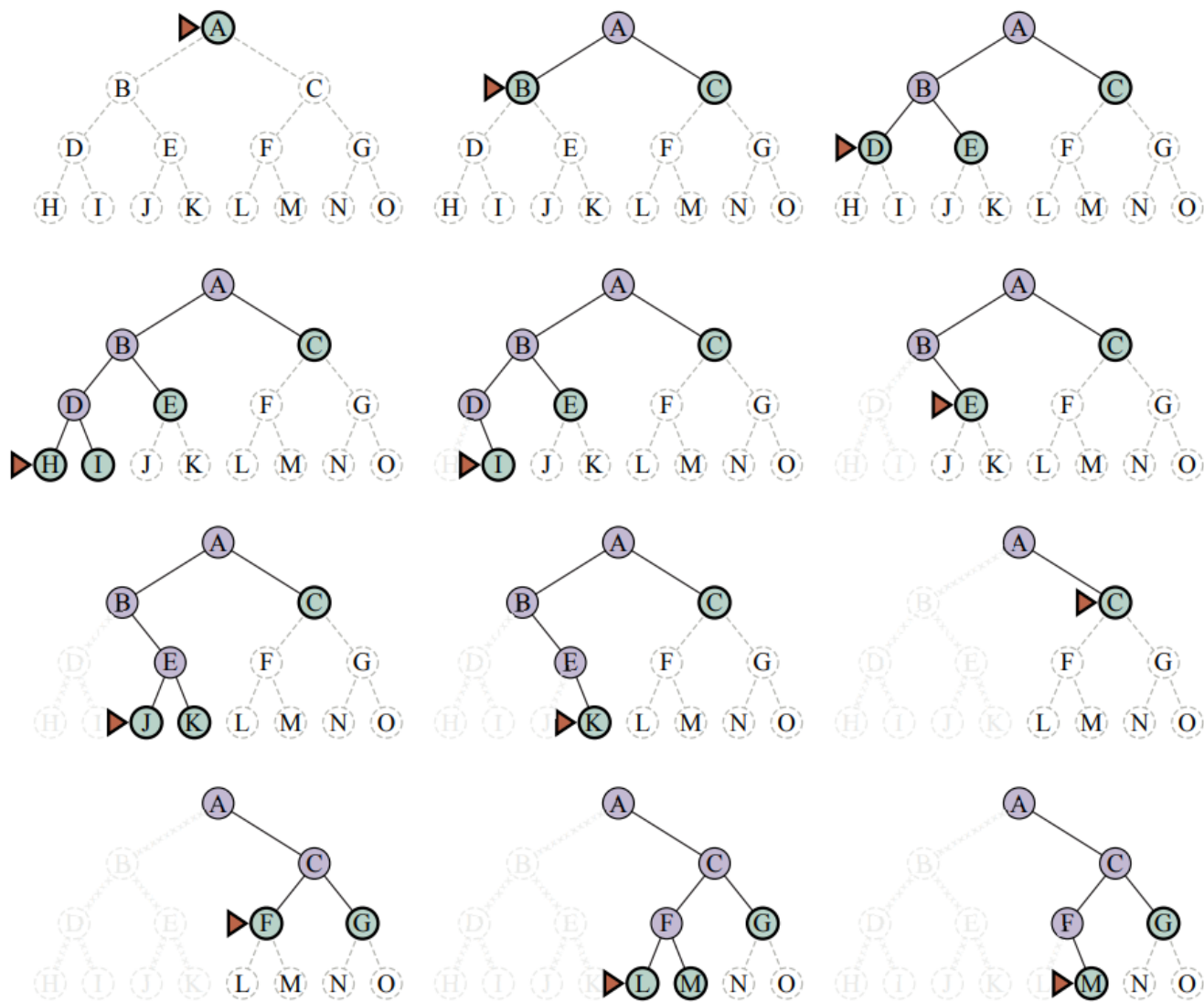
Este tipo de búsquedas, exploran el espacio de búsqueda sistemáticamente sin tener en cuenta la proximidad percibida a la meta.

Estas búsquedas se basan únicamente en la estructura del problema para encontrar una solución. Las estrategias de búsqueda no informada son: en amplitud o anchura, de costo uniforme.

Búsqueda en Profundidad

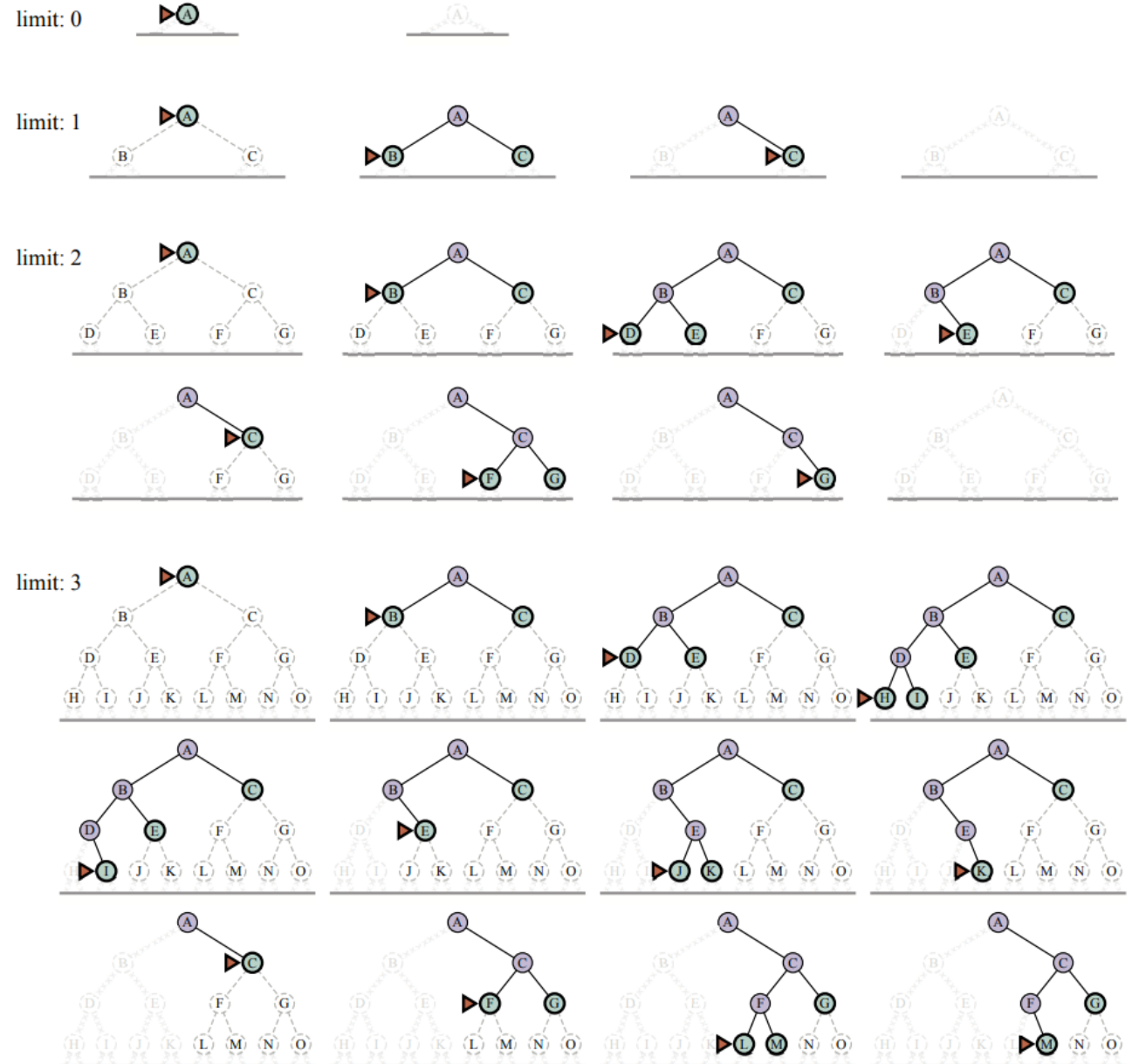
- **Depth First Search (DFS)** o búsqueda en profundidad, explora un camino lo más profundo posible en el grafo donde ocurre la búsqueda, hasta que llega a un nodo u hoja sin sucesores, o a un punto muerto. Luego, retrocede (backtracking) al nodo anterior para explorar otros caminos. Algo que se debe tener en cuenta es que no es una búsqueda óptima, puede quedarse atrapada en ciclos infinitos en el grafo. Utiliza una pila (LIFO) para gestionar los nodos a explorar.
- Para evitar los ciclos infinitos, se usa DFS con un límite máximo de profundidad (Depth Limited Search), esto permite evitar un bloqueo de la búsqueda pero, la solución no será encontrada si se encuentra más allá del límite. Esta búsqueda devuelve la primera solución encontrada, que no necesariamente es la óptima o económica en términos de tiempo y costos computacionales.

Búsqueda en Profundidad



Búsqueda en Profundidad

La implementación de DFS con profundización Iterativa (Iterative Deepening DFS - IDDFS), ejecuta la búsqueda DFS con límite de profundidad sucesivamente, comenzando con una profundidad de 0 y aumentándola en cada iteración. Esto permite un uso más eficiente de memoria. Supera los límites de la búsqueda de profundidad limitada. Aumenta el límite de profundidad paso a paso hasta encontrar una solución. Devuelve la primera solución encontrada, sin que sea necesariamente la más económica.



Búsqueda en Profundidad

function ITERATIVE-DEEPENING-SEARCH(*problem*) **returns** a solution node or *failure*
 for *depth* = 0 **to** ∞ **do**
 result $\leftarrow$ DEPTH-LIMITED-SEARCH(*problem*, *depth*)
 if *result* $\neq$ *cutoff* **then return** *result*

function DEPTH-LIMITED-SEARCH(*problem*, ℓ) **returns** a node or *failure* or *cutoff*
 frontier $\leftarrow$ a LIFO queue (stack) with NODE(*problem*.INITIAL) as an element
 result $\leftarrow$ *failure*
 while not IS-EMPTY(*frontier*) **do**
 node $\leftarrow$ POP(*frontier*)
 if *problem*.IS-GOAL(*node*.STATE) **then return** *node*
 if DEPTH(*node*) > ℓ **then**
 result $\leftarrow$ *cutoff*
 else if not IS-CYCLE(*node*) **do**
 for each *child* **in** EXPAND(*problem*, *node*) **do**
 add *child* to *frontier*
 return *result*

Búsqueda en Profundidad – EJEMPLOS

1. Búsqueda de Archivos en tu Computadora (Explorador de Windows / Linux)

El contexto: Cuando usas la barra de búsqueda de tu sistema operativo para encontrar un documento perdido (por ejemplo, tarea.pdf).

Cómo aplica DFS: El sistema no revisa primero todas las carpetas principales (eso haría BFS). En su lugar, entra a Disco C:, luego a Usuarios, luego a Documentos, luego a Universidad, luego a Semestre 1... sigue profundizando hasta que llega a una carpeta que no tiene más subcarpetas (un **punto muerto** u hoja). Si el archivo no está ahí, aplica el **backtracking**: retrocede a la carpeta anterior y entra a Semestre 2.

Por qué DFS: Es increíblemente eficiente en memoria, ya que solo necesita recordar la "ruta actual" en la que está buceando (usando una Pila/LIFO), a diferencia de BFS que colapsaría la memoria RAM de tu computadora intentando recordar todo el árbol de directorios a la vez.

Búsqueda en Profundidad – EJEMPLOS

2. Resolución de Laberintos y Sudokus (Algoritmos de Backtracking)

El contexto: Aplicaciones móviles o webs que resuelven automáticamente laberintos o juegos lógicos como el Sudoku o el problema de las N-Reinas.

Cómo aplica DFS: En un laberinto, el algoritmo toma un camino y lo sigue hasta el final. Si se topa con una pared (punto muerto), hace **backtracking**, vuelve a la última bifurcación y toma el camino alternativo. En un Sudoku, el algoritmo coloca números secuencialmente; si llega a una celda donde ningún número es válido, retrocede y borra el número anterior para intentar con otro.

Por qué DFS: Como dice el texto, DFS devuelve **la primera solución encontrada**. En un Sudoku o un laberinto, normalmente no nos importa si es "la solución más corta", simplemente nos basta con encontrar *una* solución válida.

Búsqueda en Profundidad – EJEMPLOS

3. Inteligencia Artificial en Ajedrez (Algoritmo Minimax con Límite de Profundidad)

El contexto: Los motores clásicos de ajedrez por computadora (como el famoso Deep Blue de IBM o Stockfish en sus primeras versiones) calculando su próxima jugada.

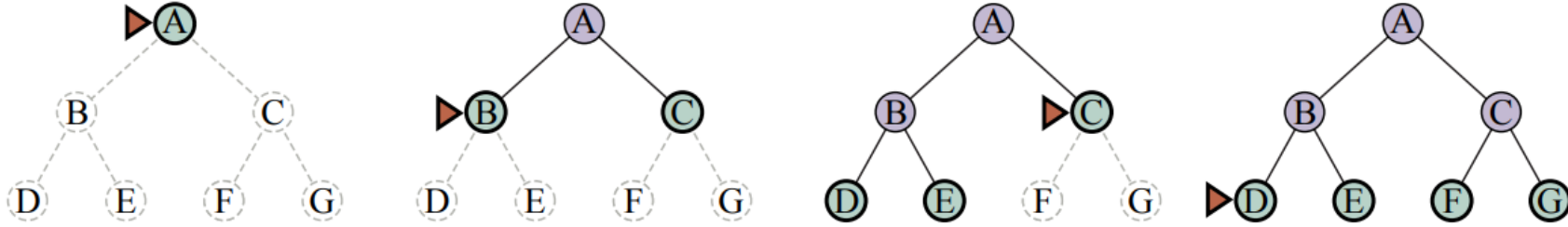
Cómo aplica DFS: La IA simula una jugada, luego la respuesta del oponente, luego su propia respuesta, profundizando en ese único escenario para ver si termina en "Jaque Mate".

Depth Limited Search: el árbol de posibilidades del ajedrez es prácticamente infinito (más jugadas posibles que átomos en el universo). Si usaran DFS normal, la IA se quedaría atascada en un **ciclo infinito** evaluando una sola línea de juego. Por eso, usan estrictamente **DFS con límite de profundidad (DLS)**, le dicen a la máquina "profundiza solo hasta 10 turnos en el futuro; detente ahí, evalúa quién va ganando y retrocede".

Búsqueda en Anchura

- La búsqueda en anchura (Breadth-First Search, BFS), explora todos los nodos de un grafo a una profundidad dada antes de pasar a la siguiente profundidad. Se trata de expandir la raíz, luego todos sus nodos secundarios y avanzar un nivel a la vez. Siempre encuentra la solución con el mínimo número de pasos desde la raíz.
- Utiliza una cola (QUEUE) para mantener el registro de los nodos a explorar, esto implica mayor uso de recursos de memoria. Expande los nodos en orden de profundidad creciente. Requiere más memoria que DFS, ya que debe almacenar todos los nodos en el nivel actual.
- Es completa para grafos finitos. Es óptima si el costo de cada paso es el mismo, por ejemplo, encontrar el camino con menos aristas. Puede ser útil cuando se necesita encontrar la solución más corta, en términos de número de pasos.

Búsqueda en Anchura



function BREADTH-FIRST-SEARCH(*problem*) **returns** a solution node or *failure*

node $\leftarrow$ NODE(*problem*.INITIAL)

if *problem*.IS-GOAL(*node*.STATE) **then return** *node*

frontier $\leftarrow$ a FIFO queue, with *node* as an element

reached $\leftarrow$ {*problem*.INITIAL}

while not IS-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

for each *child* **in** EXPAND(*problem*, *node*) **do**

s $\leftarrow$ *child*.STATE

if *problem*.IS-GOAL(*s*) **then return** *child*

if *s* is not in *reached* **then**

add *s* to *reached*

add *child* to *frontier*

return *failure*

Búsqueda en Anchura

- La búsqueda en amplitud explora el espacio de búsqueda "por niveles". Empieza en el nodo inicial y explora todos sus vecinos. Luego, explora todos los vecinos de los vecinos, y así sucesivamente.
- Utiliza una cola (FIFO, frontera de búsqueda) para mantener el orden de exploración, aquí se almacenan los nodos que serán pendientes de ser explorados, asegurando que los nodos más cercanos al inicio se exploren antes que los más lejanos.
- También se almacenarán en el conjunto "reached", los nodos que ya han sido visitados, para evitar bucles y que la exploración repita los mismos nodos.
- BFS encontrará una solución si el factor de ramificación es finito, la solución será óptima si el costo de los pasos es uniforme, es decir si todos los pasos tienen el mismo costo, BFS encontrará la solución con el menor número de pasos desde el nodo inicial.
- Esta búsqueda puede requerir mucha memoria, ya que debe almacenar todos los nodos de la frontera.

Búsqueda en Anchura

- Los problemas de búsqueda de complejidad exponencial no pueden resolverse utilizando métodos de búsqueda no informados, excepto para las instancias más pequeñas.
- Esto se puede comprender al analizar la Figura 6, para un grafo, árbol, cada nivel va a tener b^n nodos, donde b es el número de nodos hijos de cada nodo, y n es el número de nivel en el grafo (árbol), esto quiere decir, por ejemplo que en un nivel 15 se van a tener b^{15} nodos, supongamos que $b=10$, el número total de nodos sería 10^{15} , y si además cada nodo requiere 1Kb de memoria el total de memoria requerida sería 10^{15} Kb (1 millón de terabytes).
- Si se quisiera procesar 1 millón de nodos por segundo, se necesitarían 10^{12} segundos para procesar todos los nodos, el equivalente a 31.7 años (Elbadrawy, 2022).
- BFS entonces, es una herramienta útil para problemas donde el espacio de búsqueda es pequeño, su demanda de memoria limita su aplicabilidad a problemas grandes.

Problem with memory and time complexity.

If each node has 2 children:
Level 1 has 1 node
Level 2 has 2 nodes
Level 3 has 4 nodes
Level 4 has 8 nodes
Level 5 has 16 nodes



If each node has b children:
Level 1 has b^1 node
Level 2 has b^2 nodes
Level 3 has b^3 nodes
Level 4 has b^4 nodes
Level 5 has b^5 nodes

Búsqueda en Anchura – EJEMPLOS

1. Redes Sociales: Los grados de conexión (LinkedIn y Facebook)

El contexto: Cuando entras a LinkedIn, la plataforma te dice si una persona es tu contacto de "1er grado" (amigo directo), "2do grado" (amigo de tu amigo) o "3er grado".

Cómo aplica BFS: Si quieres saber cómo estás conectado con el CEO de una empresa, el algoritmo de la red social utiliza BFS. Primero busca en el Nivel 1 (todos tus amigos directos). Si no lo encuentra, expande al Nivel 2 (los amigos de tus amigos), y así sucesivamente.

Por qué BFS y no otro: Porque garantiza encontrar **la cadena de intermediarios más corta posible** (el mínimo número de pasos). Además, este es el mismo principio computacional que demuestra la famosa teoría de los "Seis grados de separación de Kevin Bacon" (verificable en *El Oráculo de Bacon* en la web).

Búsqueda en Anchura – EJEMPLOS

2. Rastreadores Web (Web Crawlers o "Spiders" de Google)

El contexto: Para que Google sepa qué páginas existen en internet y te las muestre cuando buscas algo, utiliza bots (crawlers) que navegan la web leyendo y guardando enlaces.

Cómo aplica BFS: El bot de Google empieza en una página web raíz (Nivel 0). Antes de irse a lo profundo de un solo tema, escanea **todos los enlaces de esa página** (Nivel 1) y los pone en una *Cola (Queue)*. Luego, visita las páginas del Nivel 1 y escanea sus enlaces (Nivel 2).

Por qué BFS y no otro: Hacerlo por niveles (en anchura) asegura que el motor de búsqueda indexe primero las páginas más importantes y cercanas a los sitios principales de internet, evitando quedarse atrapado infinitamente en un bucle de enlaces profundos y oscuros de una sola página. (Esto está documentado en la arquitectura básica de rastreo web).

Búsqueda en Anchura – EJEMPLOS

3. Aplicaciones de Transporte Público (Mapas de Metro o Trenes)

El contexto: Imagina el mapa del Metro de Londres, Nueva York o Madrid. Si usas la aplicación oficial para ir de una estación a otra, a menudo quieres saber la ruta que implique **el menor número de paradas o transbordos**, independientemente de la distancia física en kilómetros.

Cómo aplica BFS: Al tratar cada estación como un nodo y cada vía como una arista del mismo peso (costo = 1 paso), el software utiliza BFS para expandirse desde tu estación actual hacia las estaciones vecinas, capa por capa.

Por qué BFS y no otro: Porque, como indica la teoría, si el costo de cada paso es el mismo (1 parada de tren), BFS **es óptimo** y siempre encontrará la solución matemática con la menor cantidad de estaciones intermedias antes que cualquier otro algoritmo.

Búsqueda de Costo Uniforme

- La búsqueda de costo uniforme, toma en cuenta que cada ruta para buscar la solución, en un grafo, podría tener un coste diferente y expande los nodos de coste acumulado mínimo. De esta manera, se alcanza la solución óptima sin tener que expandir todos los nodos en cada nivel.
- La Búsqueda de Costo Uniforme es un algoritmo utilizado en inteligencia artificial para encontrar la solución de menor costo desde un nodo inicial hasta un nodo objetivo en un grafo.
- Se clasifica como una búsqueda no informada porque no utiliza heurísticas para guiar la búsqueda, sino que explora el espacio de búsqueda de manera sistemática basándose únicamente en el costo acumulado del camino.

Búsqueda de Costo Uniforme

Inicialización:

- Crear una cola de prioridad (normalmente un heap mínimo) llamada frontera.

- Insertar el nodo inicial en la frontera con un costo de 0.

- Crear un conjunto explorado para mantener un registro de los nodos que ya han sido visitados.

Iteración:

- Mientras la frontera no esté vacía:

 - Extraer el nodo de menor costo de la frontera. Llamémoslo `nodo_actual`.

 - Si `nodo_actual` es el nodo objetivo, retornar la ruta desde el nodo inicial hasta `nodo_actual`.

 - Si `nodo_actual` está en explorado, continuar con la siguiente iteración (ya lo hemos explorado por un camino más barato).

 - Añadir `nodo_actual` al conjunto explorado.

 - Para cada vecino `vecino` de `nodo_actual`:

 - Calcular el costo del camino desde el nodo inicial hasta vecino (costo del camino hasta `nodo_actual` + costo de la arista entre `nodo_actual` y vecino).

 - Si vecino no está en frontera ni en explorado, o si el nuevo costo calculado es menor que el costo actual de vecino en la frontera:

 - Insertar/actualizar vecino en la frontera con el nuevo costo.

 - Almacenar el `nodo_actual` como el padre de vecino para poder reconstruir la ruta al final.

Fallo:

- Si la frontera se vacía y no se encuentra el nodo objetivo, retornar "Fallo" (no existe una ruta).

Búsqueda de Costo Uniforme –EJEMPLOS

Aplicaciones Prácticas de la Búsqueda de Costo Uniforme (UCS). El algoritmo UCS (variante de Dijkstra) tiene un objetivo claro: **encontrar el camino que "cueste menos"**, entendiendo por "costo" cualquier recurso que queramos ahorrar (dinero, tiempo, batería, riesgo). Sus usos se agrupan en estas 3 grandes áreas:

Movimiento y Logística (GPS, Repartos y Robots):

No busca la ruta con menos pasos, sino la que consuma menos recursos. Evalúa variables sumadas como: gasolina + peajes + tiempo en el tráfico para flotas de entrega, o el gasto de batería en cada giro que da un robot para esquivar obstáculos.

Infraestructura y Producción (Redes y Fábricas):

Se usa para decidir cómo conectar cosas (ej. por dónde tirar cables de fibra óptica gastando menos material) o en qué orden deben trabajar las máquinas de una fábrica para minimizar el tiempo inactivo y los costos operativos.

Decisiones Críticas y Asignación (Medicina y Agentes):

Sirve para secuenciar acciones complejas. Por ejemplo, en medicina, calcula qué exámenes hacer primero para diagnosticar a un paciente gastando la menor cantidad de dinero y exponiéndolo al menor riesgo físico posible.



Aunque UCS es perfecto porque **siempre garantiza encontrar la solución más barata/óptima**, es un algoritmo "ciego" (no informado).

Si el mapa o el problema es gigantesco, tardará mucho tiempo en procesar porque explorará en todas las direcciones antes de decidir.

Búsqueda de Costo Uniforme –EJEMPLOS

Ejemplo de referencia. 100% real y palpable del uso de Búsqueda de Costo Uniforme / Dijkstra en la actualidad son los **Motores de Enrutamiento de Código Abierto para GPS**, específicamente el **OSRM (Open Source Routing Machine)**.

¿Qué es? Es el motor de cálculo de rutas de alto rendimiento que utilizan muchos servicios web basados en *OpenStreetMap* para dar indicaciones paso a paso (alternativo a Google Maps).

¿Cómo usa UCS/Dijkstra? Los mapas en OSRM no solo miden distancia; cada segmento de calle tiene un "costo" o "peso" basado en el límite de velocidad, si es una autopista o un camino de tierra, y si hay semáforos. El algoritmo busca el costo acumulado mínimo para trazar la ruta en milisegundos.

El sitio oficial del proyecto: [Project OSRM \(http://project-osrm.org/\)](http://project-osrm.org/). En su documentación técnica explican cómo su arquitectura base parte del algoritmo de Dijkstra (Costo Uniforme) para resolver el problema del camino más corto en grafos a escala continental.

Otro ejemplo técnico es el [Protocolo OSPF](#) que usan los routers de internet en todo el mundo de la empresa Cisco, el cual usa este mismo algoritmo para decidir por qué cable enviar un paquete de datos de forma más eficiente).



Fin de la presentación